УНИВЕРСИТЕТ ИТМО

Факультет ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине

«Проектирование вычислительных систем»

Тема: «Интерфейсы ввода-вывода общего назначения (GPIO)»
Вариант № 5 («Кодовый замок»)

Выполнили:
Студенты группы P3432
Носов Георгий Иванович
Карандашева Анастасия Денисовна

Преподаватель:
Ключев Владислав Аркадьевич

г. Санкт-Петербург
2026

Содержание

Содержание 2

1. Задание 3

Часть 1. Разработка прикладной программы 3

Часть 2. Изучение регистров GPIO 3

2. Описание логики работы программы 4

3. Архитектурное описание программы и драйверов 5

Конечный автомат опроса кнопки с подавлением дребезга 5

Конечный автомат кодового замка 6

4. Разработка собственного регистрового драйвера GPIO (Часть 2) 6

Устранение импульсных выбросов при инициализации пинов 7

Атомарность операций и групповое управление выводами 7

5. Исходные коды программы собственной разработки 9

Файл Core/Inc/my_gpio.h (Заголовочный файл регистрового драйвера GPIO) 9

Файл Core/Src/my_gpio.c (Реализация регистрового драйвера GPIO) 9

Файл Core/Inc/led.h (Заголовочный файл драйвера светодиодов) 12

Файл Core/Src/led.c (Реализация драйвера светодиодов) 11

Файл Core/Inc/button.h (Заголовочный файл драйвера кнопки) 15

Файл Core/Src/button.c (Реализация драйвера кнопки) 13

Файл Core/Inc/lock_fsm.h (Заголовочный файл автомата кодового замка) 18

Файл Core/Src/lock_fsm.c (Реализация автомата кодового замка) 15

Фрагмент главного цикла программы (main.c) 17

6. Выводы 17

1. Задание

Цели работы:

1. Получить базовые знания о принципах устройства стенда SDK-1.1M и программировании микроконтроллеров.

2. Изучить устройство интерфейсов ввода-вывода общего назначения (GPIO) в микроконтроллерах и приемы использования данных интерфейсов.

Часть 1. Разработка прикладной программы

Требуется разработать и реализовать программу для стенда SDK-1.1M со следующими функциональными возможностями:

Реализовать «кодовый замок». После ввода единственно верной последовательности не менее чем из восьми коротких и длинных нажатий должен загореться зелёный светодиод, обозначающий «открытие» замка. Светодиод горит некоторое время, потом гаснет, и система вновь переходит в начало ввода. Правильно введённый элемент последовательности должен сопровождаться единичным миганием желтого светодиода. Неправильно введенный элемент - единичным миганием красного светодиода и сбросом в начало ввода. После трёх неправильных попыток ввода красный светодиод должен мигать не менее 5 секунд, и после этого происходит сброс в начало ввода. Если код начал вводиться, но за некоторое ограниченное время не был введен до конца, желтый светодиод выдает длинный импульс и происходит сброс в начало ввода.

• **Требования к архитектуре:** функции драйверов и прикладного алгоритма должны быть строго неблокирующими. Запрещено применение задержек с активным ожиданием (функции HAL_Delay()). Многозадачность организуется в кооперативном стиле с использованием миллисекундного таймера SysTick (функция HAL_GetTick()).

Часть 2. Изучение регистров GPIO

Разработать собственную реализации драйвера GPIO (не HAL и LL) и внедрить ее в разработанную ранее программу для стенда.
Требования к драйверу:

− должны быть реализованы функции инициализации режимов работы портов ввода-вывода, получения текущих настроек режима работы порта, чтения и записи значений как единичных сигналов порта, так и одновременное нескольких сигналов в пределах одного порта;

− должна поддерживаться повторная инициализация портов с изменением режима работы, при этом не должно допускаться кратковременных «выбросов» сигналов (переключений в некорректное состояние) на выходных портах в процессе записи значений в управляющие регистры;

− поддержка прерываний не требуется;

− допускается использование определенной в HAL структуры регистров типа GPIO_TypeDef и define-определений; остальные структуры и функции для работы с GPIO использовать нельзя, требуется разработать свои варианты.

2. Описание логики работы программы

Работа программы выполняется на SDK-1.1M с на базе микроконтроллера STM32F427VIT6. Для выполнения работы, мы использовали следующие компоненты:

• **Кнопка SB1 («nBTN»):** подключена к выводу PC15. При нажатии замыкается на землю GND, соответственно, активным состоянием кнопки является логический ноль (LOW).

• **Зеленый светодиод VD6 («РАБОТА»):** подключен к выводу PD13 микроконтроллера. Логическая единица включает светодиод, логический ноль выключает.

• **Двухцветный светодиод VD7 («РЕЖИМ»):** подключен к выводам PD14 и PD15.. Управление цветом осуществляется сменой полярности напряжений на контактах PD14 и PD15.

Цвет индикатора	Состояние PD14	Состояние PD15	Назначение в задании
Желтый	1 (High, +3.3 В)	0 (Low, 0 В)	Правильный ввод элемента / Таймаут
Красный	0 (Low, 0 В)	1 (High, +3.3 В)	Ошибка ввода / Тревога блокировки
Выключен	0 (Low, 0 В)	0 (Low, 0 В)	Дежурный режим ожидания

Таблица 1. Управление двухцветным светодиодом VD7

В прошивке определен секретный пароль, состоящий из 8 элементов (комбинация коротких и длинных нажатий кнопки PC15):

Секретная последовательность: [КОРОТКОЕ, КОРОТКОЕ, ДЛИННОЕ, КОРОТКОЕ, ДЛИННОЕ, ДЛИННОЕ, КОРОТКОЕ, КОРОТКОЕ] (символически: . . - . - - . .) Проверяем нажатия с помощью измерении длительности удержания кнопки в замкнутом состоянии после программной фильтрации дребезга:

• **Короткое нажатие (SHORT / точка):** от 50 до 450 миллисекунд.

• **Длинное нажатие (LONG / тире):** 450 миллисекунд и более.

3. Архитектурное описание программы и драйверов

Архитектура программного комплекса построена по принципу модульного разделения обязанностей и кооперативной многозадачности, что было написано в задании:

• **Модуль кнопки (button.h / button.c):** реализует драйвер опроса дискретного входа PC15. Включает цифровой фильтр дребезга (порог стабильности 35 мс) и конечный автомат детекции коротких и длинных нажатий.

• **Модуль индикации (led.h / led.c):** универсальный неблокирующий драйвер светодиодов. Предоставляет сервисы одиночных таймированных вспышек и фонового периодического мигания без вызова блокирующих функций.

• **Модуль кодового замка (lock_fsm.h / lock_fsm.c):** реализует логику конечного автомата верхнего уровня: сопоставление вводимых событий с эталоном, управление таймаутом ввода и обработку блокировки при превышении порога ошибок.

• **Модуль my_gpio (my_gpio.h / my_gpio.c):** собственный драйвер регистрового уровня.

Конечный автомат опроса кнопки с подавлением дребезга

Механический контакт кнопки при замыкании и размыкании генерирует последовательность паразитных микросекундных импульсов. Для предотвращения ложных срабатываний реализован конечный автомат, диаграмма состояний которого описана ниже:

• **FSM_BTN_IDLE:** исходное состояние покоя (кнопка разомкнута, уровень HIGH). При обнаружении уровня LOW фиксируется потенциальное нажатие и осуществляется переход в состояние FSM_BTN_DEBOUNCE_DOWN с запоминанием стартовой временной метки.

• **FSM_BTN_DEBOUNCE_DOWN:** ожидание истечения интервала фильтрации дребезга (35 мс). Если по истечении интервала уровень остается LOW, нажатие признается валидным, фиксируется метка времени нажатия и автомат переходит в FSM_BTN_DOWN.

• **FSM_BTN_DOWN:** состояние удержания кнопки. Ожидается переход входного сигнала в уровень HIGH (начало отпускания кнопки).

• **FSM_BTN_DEBOUNCE_UP:** фильтрация дребезга при размыкании (35 мс). При стабильном высоком уровне вычисляется суммарная длительность удержания: при длительности >= 450 мс генерируется событие BTN_EVENT_LONG, иначе (от 50 до 450 мс) – BTN_EVENT_SHORT. Возврат в FSM_BTN_IDLE.

Конечный автомат кодового замка

Прикладная логика кодового замка представлена конечным автоматом со следующими состояниями:

• **LOCK_STATE_INPUT:** прием и валидация очередного нажатия. При бездействии пользователя свыше 5 секунд формируется длинный импульс желтого светодиода и сброс позиции ввода. При ошибке – вспышка красного, при 3 ошибках – переход в режим блокировки.

• **LOCK_STATE_UNLOCKED:** замок открыт. Зелёный светодиод горит в течение 3 секунд. Нажатия кнопки на этот период игнорируются.

• **LOCK_STATE_ALARM_BLOCKED:** режим тревоги после трех ошибок. Красный светодиод циклически мигает с периодом 200 мс в течение 5.5 секунд. Ввод блокирован.

• **LOCK_STATE_TIMEOUT_PULSE:** формирование длинного импульса индикатора таймаута (1.5 с).

4. Разработка собственного регистрового драйвера GPIO (Часть 2)

По 2 части задания, мы разработали драйвер прямого доступа к аппаратным регистрам контроллера GPIO. Драйвер оперирует стандартной базовой структурой GPIO_TypeDef, определенной в заголовочных файлах CMSIS.

Регистр	Разрядность / Доступ	Назначение и функциональная роль
MODER	32 бит (RW)	Режим работы вывода (2 бита на пин): 00 – вход, 01 – выход общего назначения, 10 – альтернативная функция, 11 – аналоговый режим.
OTYPER	32 бит (RW)	Тип выходного каскада (1 бит на пин): 0 – двухтактный выход (Push-Pull), 1 – выход с открытым стоком (Open-Drain).
OSPEEDR	32 бит (RW)	Скорость нарастания фронтов выходного сигнала: 00 – 2 МГц, 01 – 25 МГц, 10 – 50 МГц, 11 – 100 МГц.
PUPDR	32 бит (RW)	Конфигурация внутренних подтягивающих резисторов: 00 – отключены, 01 – подтяжка к VDD (Pull-Up), 10 – подтяжка к GND (Pull-Down).

IDR	32 бит (RO)	Входные данные порта. Младшие 16 бит отражают фактические логические уровни напряжения на физических контактах микроконтроллера.
ODR	32 бит (RW)	Выходные данные порта. Определяет уровни напряжения, выдаваемые выходными буферами.
BSRR	32 бит (WO)	Атомарная установка и сброс бит выходов: биты 0..15 устанавливают соответствующие биты в 1, биты 16..31 сбрасывают биты в 0.

Таблица 2. Назначение используемых регистров GPIO в микроконтроллерах STM32F4

Устранение импульсных выбросов при инициализации пинов

По требованию задания надо было исключить кратковременные переключения на выходных портах при их повторной инициализации. В стандартных реализациях, если пин сначала переводится в режим выхода в регистре MODER, на контакте мгновенно устанавливается старое значение из регистра ODR. Если требуемый начальный уровень отличается от текущего, на линии возникает кратковременный всплеск напряжения.

В разработанной функции my_gpio_init данная проблема решена аппаратным опережением: требуемый логический уровень (init_state) записывается в регистр BSRR ДО ТОГО, как соответствующее поле регистра MODER переводится в режим выхода:

```
if (config->mode == MY_GPIO_MODE_OUTPUT_PP || config->mode == MY_GPIO_MODE_OUTPUT_OD) {
    if (config->init_state) {
        GPIOx->BSRR = pin;          /* Заранее подготовлена 1 */
    } else {
        GPIOx->BSRR = (pin << 16U);  /* Заранее подготовлен 0 */
    }
}
/* Только после этого активируется режим выхода в MODER */
GPIOx->MODER &= ~(0x3U << (pos * 2U));
GPIOx->MODER |= (mode_val << (pos * 2U));
```

В результате в момент активации выходного буфера на пине уже присутствует целевой потенциал, что гарантирует абсолютное отсутствие коммутационных помех.

Атомарность операций и групповое управление выводами

Для управления выходами в драйвере используется исключительно регистр BSRR, что обеспечивает атомарность операции за один машинный такт процессора без применения

операций «чтение-модификация-запись» (R-M-W) и без необходимости глобального запрета прерываний.

Для синхронного переключения состояния двухцветного светодиода реализована функция my_gpio_write_port, выполняющая раздельную установку и сброс выводов по маске в одной 32-битной транзакции:

```
void my_gpio_write_port(GPIO_TypeDef *GPIOx, uint16_t mask, uint16_t value) {
    uint32_t set_bits = (uint32_t)(mask & value);
    uint32_t reset_bits = (uint32_t)(mask & (~value));
    GPIOx->BSRR = set_bits | (reset_bits << 16U);
}
```

Результаты практического тестирования программы на стенде SDK-1.1M

После компиляции проекта прошивка была загружена во флеш-память стенда SDK-1.1M с помощью встроенного отладчика OpenOCD. В ходе практических испытаний были проверены все предусмотренные вариантом задания режимы работы устройства.

На фотографиях ниже представлена фиксация работы индикации на боковой панели стенда в различных состояниях конечного автомата:

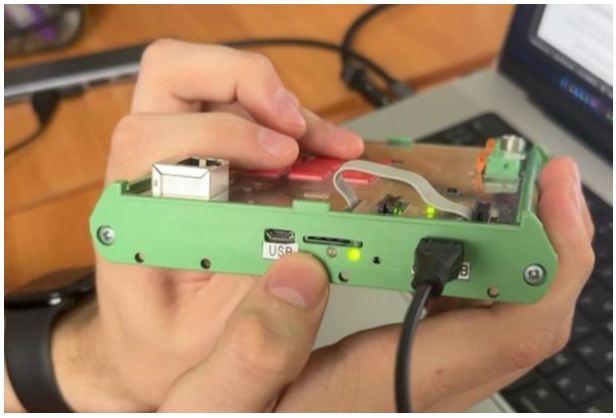


Рисунок 1. Индикация работы программы на стенде SDK-1.1M: успешное открытие замка (зеленый светодиод VD6)

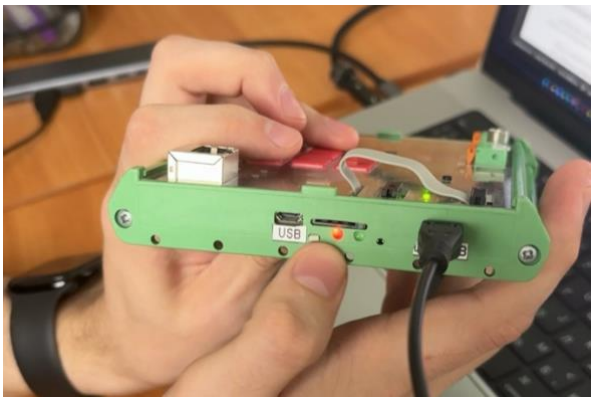


Рисунок 2. Индикация ошибки ввода и перехода в режим блокировки (красный светодиод VD7)

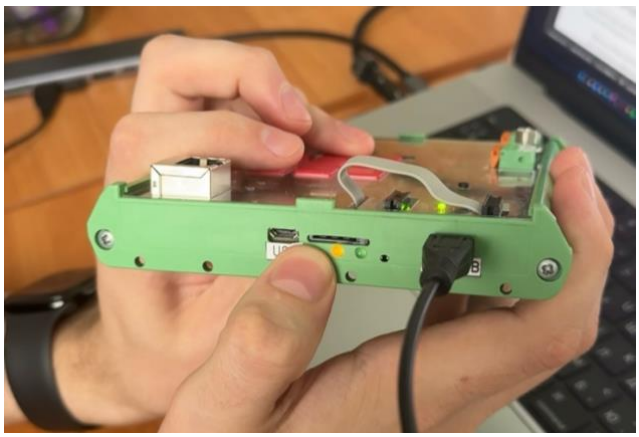


Рисунок 3. Индикация ввода элементов кодовой последовательности и сброса по таймауту (желтый светодиод VD7)

5. Исходные коды программы собственной разработки

В соответствии с методическими требованиями, ниже приведены исходные коды исключительно собственной разработки (файлы драйверов, логики и прикладного цикла), без включения кода библиотек и автогенератора CubeMX.

Файл Core/Src/my_gpio.c (Реализация регистрового драйвера GPIO)

```
#include "my_gpio.h"

void my_gpio_init(GPIO_TypeDef *GPIOx, const MyGPIOConfig *config)
{
    if (GPIOx == NULL || config == NULL) {
        return;
    }

    for (uint32_t pos = 0; pos < 16; pos++) {
        uint16_t pin = (1U << pos);
        if (config->pin_mask & pin) {
            /* предварительно выставляем начальный уровень через BSRR. */
            if (config->mode == MY_GPIO_MODE_OUTPUT_PP || config->mode == MY_GPIO_MODE_OUTPUT_OD) {
                if (config->init_state) {
                    GPIOx->BSRR = pin;          /* установка 1 */
                } else {
                    GPIOx->BSRR = (pin << 16U);  /* сброс в 0 */
                }
            }

            /* настройка типа выхода (Push-Pull / Open-Drain) */
            if (config->mode == MY_GPIO_MODE_OUTPUT_OD) {
                GPIOx->OTYPER |= (1U << pos);
            } else {
                GPIOx->OTYPER &= ~(1U << pos);
            }

            /* настройка скорости порта (OSPEEDR, 2 бита на пин) */
            GPIOx->OSPEEDR &= ~(0x3U << (pos * 2U));
            GPIOx->OSPEEDR |= ((uint32_t)config->speed << (pos * 2U));

            /* настройка подтяжки (PUPDR, 2 бита на пин) */
            GPIOx->PUPDR &= ~(0x3U << (pos * 2U));
            GPIOx->PUPDR |= ((uint32_t)config->pull << (pos * 2U));
```

```
        /* настройка режима работы (MODER, 2 бита на пин) */
        uint32_t mode_val = 0;
        switch (config->mode) {
            case MY_GPIO_MODE_INPUT:     mode_val = 0x0U; break;
            case MY_GPIO_MODE_OUTPUT_PP:
            case MY_GPIO_MODE_OUTPUT_OD: mode_val = 0x1U; break;
            case MY_GPIO_MODE_ANALOG:    mode_val = 0x3U; break;
            default:                     mode_val = 0x0U; break;
        }
        GPIOx->MODER &= ~(0x3U << (pos * 2U));
        GPIOx->MODER |= (mode_val << (pos * 2U));
      }
    }
}

void my_gpio_get_config(GPIO_TypeDef *GPIOx, uint16_t pin, MyGPIOConfig *config)
{
    if (GPIOx == NULL || config == NULL || pin == 0) {
        return;
    }

    uint32_t pos = 0;
    while ((pin & (1U << pos)) == 0 && pos < 16) {
        pos++;
    }
    if (pos >= 16) return;

    config->pin  mask = (1U << pos);

    uint32_t moder = (GPIOx->MODER >> (pos * 2U)) & 0x3U;
    uint32_t otyper = (GPIOx->OTYPER >> pos) & 0x1U;
    if (moder == 0x0U) {
        config->mode = MY_GPIO_MODE_INPUT;
    } else if (moder == 0x1U) {
        config->mode = (otyper == 1U) ? MY_GPIO_MODE_OUTPUT_OD : MY_GPIO_MODE_OUTPUT_PP;
    } else if (moder == 0x3U) {
        config->mode = MY_GPIO_MODE_ANALOG;
    }

    config->pull = (MyGPIOPull)((GPIOx->PUPDR >> (pos * 2U)) & 0x3U);
    config->speed = (MyGPIOSpeed)((GPIOx->OSPEEDR >> (pos * 2U)) & 0x3U);
    config->init_state = (uint8_t)((GPIOx->ODR >> pos) & 0x1U);
}

void my_gpio_write_pin(GPIO_TypeDef *GPIOx, uint16_t pin, uint8_t state)
{
    if (GPIOx == NULL) return;

    if (state) {
        GPIOx->BSRR = (uint32_t)pin;           /* Биты BS0..BS15 */
    } else {
        GPIOx->BSRR = ((uint32_t)pin << 16U);   /* Биты BR0..BR15 */
    }
}

uint8_t my_gpio_read_pin(GPIO_TypeDef *GPIOx, uint16_t pin)
{
    if (GPIOx == NULL) return 0;
    return ((GPIOx->IDR & pin) != 0U) ? 1U : 0U;
}

void my_gpio_toggle_pin(GPIO_TypeDef *GPIOx, uint16_t pin)
{
    if (GPIOx == NULL) return;
    uint32_t odr = GPIOx->ODR;
    GPIOx->BSRR = ((odr & pin) << 16U) | (~odr & pin);
}
```

```
void my_gpio_write_port(GPIO_TypeDef *GPIOx, uint16_t mask, uint16_t value)
{
    if (GPIOx == NULL) return;
    /* Атомарная одновременная запись нескольких битов порта */
    uint32_t set_bits = (uint32_t)(mask & value);
    uint32_t reset_bits = (uint32_t)(mask & (~value));
    GPIOx->BSRR = set_bits | (reset_bits << 16U);
}

uint16_t my_gpio_read_port(GPIO_TypeDef *GPIOx, uint16_t mask)
{
    if (GPIOx == NULL) return 0;
    return (uint16_t)(GPIOx->IDR & mask);
}
```

Файл Core/Src/led.c (Реализация драйвера светодиодов)

```
#include "led.h"
#include "my_gpio.h"
#include "stm32f4xx_hal.h"

#define LED_PIN_GREEN   GPIO_PIN_13
#define LED_PIN_YEL     GPIO_PIN_14
#define LED_PIN_RED     GPIO_PIN_15
#define LED_GPIO_PORT   GPIOD

/* Структура для управления одиночной вспышкой */
typedef struct {
    bool     active;
    LedId    led;
    uint32_t end_tick;
} FlashEffect;

/* Структура для циклического мигания */
typedef struct {
    bool     active;
    LedId    led;
    uint32_t toggle_period_ms;
    uint32_t next_toggle_tick;
    uint32_t end_tick;
    bool     current_state;
} BlinkEffect;

static FlashEffect s_flash = {0};
static BlinkEffect s_blink = {0};

/* Низкоуровневая запись пинов (с поддержкой выбора: my_gpio или HAL) */
static void hw_write_leds(bool green, bool yel, bool red)
{
#if USE_CUSTOM_GPIO
    my_gpio_write_pin(LED_GPIO_PORT, LED_PIN_GREEN, green ? 1 : 0);
    uint16_t mask = LED_PIN_YEL | LED_PIN_RED;
    uint16_t val = 0;
    if (yel) {
        val = LED_PIN_YEL; /* PD14 = 1, PD15 = 0 */
    } else if (red) {
        val = LED_PIN_RED; /* PD14 = 0, PD15 = 1 */
    }
    my_gpio_write_port(LED_GPIO_PORT, mask, val);
#else
    HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_GREEN, green ? GPIO_PIN_SET : GPIO_PIN_RESET);
    if (yel) {
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_YEL, GPIO_PIN_SET);
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_RED, GPIO_PIN_RESET);
    } else if (red) {
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_YEL, GPIO_PIN_RESET);
```

```
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_RED, GPIO_PIN_SET);
    } else {
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_YEL, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(LED_GPIO_PORT, LED_PIN_RED, GPIO_PIN_RESET);
    }
#endif
}

static bool s_cur_green = false;
static bool s_cur_yel   = false;
static bool s  cur  red   = false;

void led_init(void)
{
#if USE_CUSTOM_GPIO
    __HAL_RCC_GPIOD_CLK_ENABLE();
    MyGPIOConfig cfg = {
        .pin_mask   = LED_PIN_GREEN | LED_PIN_YEL | LED_PIN_RED,
        .mode       = MY_GPIO_MODE_OUTPUT_PP,
        .pull       = MY_GPIO_PULL_NONE,
        .speed      = MY_GPIO_SPEED_LOW,
        .init_state = 0
    };
    my_gpio_init(LED_GPIO_PORT, &cfg);
#endif
    led_set(LED_ALL, LED_STATE_OFF);
}

void led_set(LedId led, LedState state)
{
    bool on = (state == LED_STATE_ON);
    switch (led) {
        case LED_GREEN:
            s_cur_green = on;
            break;
        case LED_YELLOW:
            s_cur_yel = on;
            if (on) s_cur_red = false;
            break;
        case LED_RED:
            s_cur_red = on;
            if (on) s_cur_yel = false;
            break;
        case LED_ALL:
            s_cur_green = on;
            s_cur_yel   = false;
            s_cur_red   = false;
            break;
    }
    hw_write_leds(s_cur_green, s_cur_yel, s_cur_red);
}

void led_trigger_flash(LedId led, uint32_t duration_ms)
{
    led_stop_effects();
    led_set(led, LED_STATE_ON);
    s_flash.active   = true;
    s_flash.led      = led;
    s_flash.end_tick = HAL_GetTick() + duration_ms;
}

void led_start_blink(LedId led, uint32_t toggle_period_ms, uint32_t total_duration_ms)
{
    led_stop_effects();
    s_blink.active            = true;
    s_blink.led               = led;
    s_blink.toggle_period_ms  = toggle_period_ms;
    s_blink.current_state     = true;
    s_blink.next_toggle_tick  = HAL_GetTick() + toggle_period_ms;
```

```
    s_blink.end_tick          = HAL_GetTick() + total_duration_ms;

    led_set(led, LED_STATE_ON);
}

bool led_is_blinking(void)
{
    return s_blink.active;
}

void led_stop_effects(void)
{
    if (s_flash.active) {
        led_set(s_flash.led, LED_STATE_OFF);
        s_flash.active = false;
    }
    if (s_blink.active) {
        led_set(s_blink.led, LED_STATE_OFF);
        s_blink.active = false;
    }
}

void led_update(uint32_t current_tick)
{
    /* Обновление циклического мигания */
    if (s_blink.active) {
        if ((int32_t)(current_tick - s_blink.end_tick) >= 0) {
            /* Время мигания истекло */
            led_set(s_blink.led, LED_STATE_OFF);
            s_blink.active = false;
        } else if ((int32_t)(current_tick - s_blink.next_toggle_tick) >= 0) {
            s_blink.current_state = !s_blink.current_state;
            led_set(s_blink.led, s_blink.current_state ? LED_STATE_ON : LED_STATE_OFF);
            s_blink.next_toggle_tick = current_tick + s_blink.toggle_period_ms;
        }
        return;
    }

    /* Обновление одиночной вспышки */
    if (s_flash.active) {
        if ((int32_t)(current_tick - s_flash.end_tick) >= 0) {
            led_set(s_flash.led, LED_STATE_OFF);
            s_flash.active = false;
        }
    }
}
```

Файл Core/Src/button.c (Реализация драйвера кнопки)

```
#include "button.h"
#include "my_gpio.h"
#include "stm32f4xx_hal.h"

#define BTN_PIN             GPIO_PIN_15
#define BTN_PORT            GPIOC

#define DEBOUNCE_TIME_MS    35U   /* Время фильтрации дребезга контактов */
#define MIN_PRESS_TIME_MS   50U   /* Минимальная длительность валидного нажатия */
#define LONG_PRESS_TIME_MS  450U  /* Порог разделения на короткое и длинное */

typedef enum {
    FSM_BTN_IDLE = 0,
    FSM_BTN_DEBOUNCE_DOWN,
    FSM_BTN_DOWN,
    FSM_BTN_DEBOUNCE_UP
} BtnFsmState;
```

```
static BtnFsmState s_state = FSM_BTN_IDLE;
static uint32_t s_state_start_tick = 0;
static uint32_t s_press_start_tick = 0;

static bool hw_read_button_pressed(void)
{
#if USE_CUSTOM_GPIO
   return (my_gpio_read_pin(BTN_PORT, BTN_PIN) == 0U);
#else
   return (HAL_GPIO_ReadPin(BTN_PORT, BTN_PIN) == GPIO_PIN_RESET);
#endif
}

void button_init(void)
{
#if USE_CUSTOM_GPIO
   __HAL_RCC_GPIOC_CLK_ENABLE();
   MyGPIOConfig cfg = {
      .pin_mask   = BTN_PIN,
      .mode       = MY_GPIO_MODE_INPUT,
      .pull       = MY_GPIO_PULL_UP,
      .speed      = MY_GPIO_SPEED_LOW,
      .init_state = 0
   };
   my_gpio_init(BTN_PORT, &cfg);
#endif
   s_state = FSM_BTN_IDLE;
}

ButtonEvent button_update(uint32_t current_tick)
{
   ButtonEvent event = BTN_EVENT_NONE;
   bool pressed = hw_read_button_pressed();

   switch (s_state) {
      case FSM_BTN_IDLE:
         if (pressed) {
            s_state = FSM_BTN_DEBOUNCE_DOWN;
            s_state_start_tick = current_tick;
         }
         break;

      case FSM_BTN_DEBOUNCE_DOWN:
         if ((current_tick - s_state_start_tick) >= DEBOUNCE_TIME_MS) {
            if (pressed) {
               /* Дребезг прошёл */
               s_state = FSM_BTN_DOWN;
               s_press_start_tick = current_tick;
            } else {
               s_state = FSM_BTN_IDLE;
            }
         }
         break;

      case FSM_BTN_DOWN:
         if (!pressed) {
            s_state = FSM_BTN_DEBOUNCE_UP;
            s_state_start_tick = current_tick;
         }
         break;

      case FSM_BTN_DEBOUNCE_UP:
         if ((current_tick - s_state_start_tick) >= DEBOUNCE_TIME_MS) {
            if (!pressed) {
               /* Кнопка отпущена, считаем длительность нажатия */
               uint32_t duration = current_tick - s_press_start_tick;
               if (duration >= LONG_PRESS_TIME_MS) {
                  event = BTN_EVENT_LONG;
```

```
            } else if (duration >= MIN_PRESS_TIME_MS) {
                event = BTN_EVENT_SHORT;
            }
            s_state = FSM_BTN_IDLE;
        } else {
            /* Ложное размыкание, возвращаемся в состояние удержания */
            s_state = FSM_BTN_DOWN;
        }
    }
    break;
  }

  return event;
}
```

Файл Core/Src/lock_fsm.c (Реализация автомата кодового замка)

```
#include "lock_fsm.h"
#include "led.h"
#include <stdbool.h>

#define CODE_LENGTH        8U
#define INPUT_TIMEOUT_MS   5000U  /* Таймаут между нажатиями */
#define UNLOCKED_TIME_MS   3000U  /* Время горения зелёного при открытии */
#define ALARM_TIME_MS      5500U  /* Время мигания красного при 3 ошибках */
#define TIMEOUT_PULSE_MS   1500U  /* Длинный импульс жёлтого при таймауте */
#define FLASH_YELLOW_MS    200U   /* Короткая вспышка жёлтого при правильном элементе */
#define FLASH_RED_MS       250U   /* Вспышка красного при ошибке */

/* Секретная комбинация:
 * К, К, Д, К, Д, Д, К, К */
static const ButtonEvent SECRET_CODE[CODE_LENGTH] = {
  BTN_EVENT_SHORT,
  BTN_EVENT_SHORT,
  BTN_EVENT_LONG,
  BTN_EVENT_SHORT,
  BTN_EVENT_LONG,
  BTN_EVENT_LONG,
  BTN_EVENT_SHORT,
  BTN_EVENT_SHORT
};

typedef enum {
  LOCK_STATE_INPUT = 0,
  LOCK_STATE_UNLOCKED,
  LOCK_STATE_ALARM_BLOCKED,
  LOCK_STATE_TIMEOUT_PULSE
} LockState;

static LockState s_lock_state = LOCK_STATE_INPUT;
static uint8_t   s_input_index = 0;
static uint8_t   s_error_count = 0;
static uint32_t  s_last_press_tick = 0;
static uint32_t  s_state_end_tick = 0;

void lock_fsm_init(void)
{
  s_lock_state     = LOCK_STATE_INPUT;
  s_input_index    = 0;
  s_error_count    = 0;
  s_last_press_tick = 0;
  s_state_end_tick  = 0;
  led_set(LED_ALL, LED_STATE_OFF);
}

void lock_fsm_update(ButtonEvent btn_event, uint32_t current_tick)
```

```
{
    switch (s_lock_state) {
        case LOCK_STATE_INPUT:
            if (s_input_index > 0) {
                if ((current_tick - s_last_press_tick) >= INPUT_TIMEOUT_MS) {
                    /* Таймаут истёк: длинный импульс жёлтого и сброс */
                    s_input_index = 0;
                    led_trigger_flash(LED_YELLOW, TIMEOUT_PULSE_MS);
                    s_lock_state = LOCK_STATE_TIMEOUT_PULSE;
                    s_state_end_tick = current_tick + TIMEOUT_PULSE_MS;
                    return;
                }
            }

            /* Обработка поступившего нажатия */
            if (btn_event != BTN_EVENT_NONE) {
                s_last_press_tick = current_tick;

                if (btn_event == SECRET_CODE[s_input_index]) {
                    /* Правильный элемент */
                    s_input_index++;

                    if (s_input_index >= CODE_LENGTH) {
                        /* Вся последовательность введена верно! */
                        s_input_index = 0;
                        s_error_count = 0;
                        led_stop_effects();
                        led_set(LED_GREEN, LED_STATE_ON);
                        s_lock_state = LOCK_STATE_UNLOCKED;
                        s_state_end_tick = current_tick + UNLOCKED_TIME_MS;
                    } else {
                        led_trigger_flash(LED_YELLOW, FLASH_YELLOW_MS);
                    }
                } else {
                    /* Неправильный элемент */
                    s_input_index = 0;
                    s_error_count++;

                    if (s_error_count >= 3) {
                        /* 3 неверных попытки подряд - тревога */
                        s_error_count = 0;
                        led_start_blink(LED_RED, 200, ALARM_TIME_MS);
                        s_lock_state = LOCK_STATE_ALARM_BLOCKED;
                        s_state_end_tick = current_tick + ALARM_TIME_MS;
                    } else {
                        led_trigger_flash(LED_RED, FLASH_RED_MS);
                    }
                }
            }
            break;

        case LOCK_STATE_UNLOCKED:
            /* Замок открыт, нажатия игнорируются */
            if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
                led_set(LED_GREEN, LED_STATE_OFF);
                s_lock_state = LOCK_STATE_INPUT;
            }
            break;

        case LOCK_STATE_ALARM_BLOCKED:
            /* Блокировка, нажатия игнорируются */
            if ((int32_t)(current_tick - s_state_end_tick) >= 0 && !led_is_blinking()) {
                s_lock_state = LOCK_STATE_INPUT;
            }
            break;

        case LOCK_STATE_TIMEOUT_PULSE:
            if ((int32_t)(current_tick - s_state_end_tick) >= 0) {
                s_lock_state = LOCK_STATE_INPUT;
```

```
        }
        break;
    }
}
```

Фрагмент главного цикла программы (main.c)

```
int main(void)
{
 HAL_Init();
 SystemClock_Config();
 MX_GPIO_Init();

 /* Инициализация пользовательских модулей */
 led_init();
 button_init();
 lock_fsm_init();

 /* Главный неблокирующий цикл программы */
 while (1)
 {
  uint32_t now = HAL_GetTick();

  /*  Опрос кнопки (антидребезг, классификация SHORT / LONG) */
  ButtonEvent evt = button_update(now);

  /*  Обновление автомата прикладной логики кодового замка */
  lock_fsm_update(evt, now);

  /* Обновление светодиодов (вспышки, таймированные мигания) */
  led_update(now);
 }
}
```

6. Выводы

В ходе выполнения работы мы освоили базовые принципы работы со стендом и портами GPIO на микроконтроллере STM32.

Научились правильно считывать нажатия с кнопок с программной защитой от дребезга, управлять световой индикацией и организовывать логику программы без использования блокирующих задержек, чтобы всё работало параллельно и без подвисаний. Дополнительно разобрались с тем, как настраивать выводы напрямую через регистры процессора без использования HAL. Программу успешно загрузили на стенд и убедились в её корректной работе.